Linear Regression in R

Daniel DellaPosta

June 29, 2022

Outline

Data Preparation

- Data for regression analysis should be in "data frame" object
- Cases are in rows, accessible by *data name[number,]*
- Variables are in columns, accessible by *data name[, number]* or *data name$variable name*
- If unsure, check with "is.data.frame(*data name*)"
- Make sure that outcome variable is coded as numeric (*is.numeric(data name$outcome variable*)

Outline

Running Regressions

- Your basic, no-frills linear (OLS) regression in R:

  lm(outcome variable $\sim$ *predictor variable 1* $+$ *predictor variable 2* $+$ *... + last predictor variable, data = data name)*
- To simply display the regression results in R, use *summary(lm(...))* (see above for example of what is typically included in the "..." here and below)
- To save regression as an object (which I recommend), pick a name for the new object and do *object name = lm(...)*

Example

- For Mafia members, how is number of network ties (degree) associated with age (agecen), ethnicity (nonital), membership in a NYC family (nyfive), and kin connections to other Mafia members (kin)?

```
> reg = lm(degree ~ agecen + nonital + nyfive + kin, data=d)
> summary(reg)

Call:
lm(formula = degree ~ agecen + nonital + nyfive + kin, data = d)

Residuals:
    Min      1Q  Median      3Q     Max
-11.483  -3.608  -1.180   1.890  66.904

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  5.98141    0.39405  15.179  < 2e-16 ***
agecen       0.07134    0.02434   2.931  0.00349 **
nonital     -0.93336    1.30628  -0.715  0.47514
nyfive       1.57365    0.48991   3.212  0.00138 **
kin          1.92054    0.21252   9.037  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 6.253 on 702 degrees of freedom
Multiple R-squared:  0.1155,    Adjusted R-squared:  0.1105
F-statistic: 22.92 on 4 and 702 DF,  p-value: < 2.2e-16
```

Running Regressions II

- Many transformations of variables can be done within *lm(...)*, without need to even "recode" variable in data frame
- In the example below, the outcome variable is logged and there is an interaction effect between two predictors

```
> reg2 = lm(log(degree) ~ agecen + nonital+nyfive*kin, data=d)
> summary(reg2)

Call:
lm(formula = log(degree) ~ agecen + nonital + nyfive * kin, data = d)

Residuals:
     Min       1Q   Median       3Q      Max
-2.00807 -0.41757  0.08368  0.42798  2.09509

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.690069   0.045515  37.132  < 2e-16 ***
agecen       0.006443   0.002707   2.380  0.01758 *
nonital     -0.078347   0.144989  -0.540  0.58912
nyfive      -0.004575   0.061111  -0.075  0.94034
kin          0.157477   0.028276   5.569 3.65e-08 ***
nyfive:kin   0.147085   0.051366   2.863  0.00432 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.694 on 701 degrees of freedom
Multiple R-squared:  0.112,     Adjusted R-squared:  0.1056
F-statistic: 17.68 on 5 and 701 DF,  p-value: < 2.2e-16
```

Outline

Post-Estimation

- If you save the regression model as an object (as shown earlier), you can use that saved object for post-estimation commands
- For example, use *predict(regression object)* to inspect fitted values from the model (you can also create a new variable in the data frame from these)
- Inspect residuals stored in *regression object$residuals*
- R also has many options for evaluating model fit, such as *BIC(regression object)* or *AIC(regression object)*, or for comparing models, such as *anova(regression object 1, regression object 2)*

Outline

Going Further

- Options for more advanced statistical approaches in R are almost literally endless
- A good place to begin expanding your knowledge would be with the *glm()* function, which allows for logistic regression and other specifications

```
> #GLM example
> reg3 = glm(apalachin ~ agecen + nonital + nyfive + kin, data = d, family="binomial")
> summary(reg3)

Call:
glm(formula = apalachin ~ agecen + nonital + nyfive + kin, family = "binomial",
    data = d)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-0.6852  -0.5065  -0.3622  -0.3086   2.5705

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  -1.93285    0.19536  -9.894   <2e-16 ***
agecen        0.02535    0.01406   1.803   0.0713 .
nonital     -15.18035  793.69276  -0.019   0.9847
nyfive       -0.85469    0.28795  -2.968   0.0030 **
kin          -0.01734    0.11178  -0.155   0.8767
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 420.20  on 706  degrees of freedom
Residual deviance: 400.33  on 702  degrees of freedom
AIC: 410.33

Number of Fisher Scoring iterations: 16
```

Going Further II

- Most statistical models beyond basic regression will be contained in R packages that you'll need to install before using
- How do you find an R package for what you're trying to do? In my experience, Google is your friend...
- Often there will be several available packages for any given method
- For example, if you are using multilevel mixed-effects models, there's *lme4*, *nlme*, *MCMCglmm*, and others available
- Every function will have its own rules and specifications, so my "Golden Rule" is to always check these (*?function name*) before using a new one